Open-Source Component Inventory

IINoT Lone Worker Safety — Blockchain India Challenge Submission

Document purpose: Satisfies the Blockchain India Challenge requirement that "all open-source components used must be clearly identified and documented in the submission, including version numbers and licenses."

---

1. Blockchain Infrastructure

These components form the permissioned blockchain network layer. All are published by the Linux Foundation's LF Decentralized Trust (formerly Hyperledger) project.

Component	Version	License	Role in solution
Hyperledger Fabric peer binary	v2.5.15	Apache 2.0	Peer node — maintains ledger copy, executes chaincode, endorses transactions
Hyperledger Fabric orderer binary	v2.5.15	Apache 2.0	Ordering service — sequences transactions into blocks using Raft consensus
Hyperledger Fabric CA binary	v1.5.17	Apache 2.0	Certificate authority — issues X.509 identities with role attributes to all participants
Hyperledger Fabric tools binary	v2.5.15	Apache 2.0	CLI tools — `peer`, `configtxgen`, `osnadmin` for network administration
hyperledger/fabric-peer (Docker)	2.5.15	Apache 2.0	Peer container image
hyperledger/fabric-orderer (Docker)	2.5.15	Apache 2.0	Orderer container image
hyperledger/fabric-ca (Docker)	latest	Apache 2.0	Fabric CA container image
hyperledger/fabric-tools (Docker)	latest	Apache 2.0	Admin CLI container image
hyperledger/fabric-nodeenv (Docker)	2.5	Apache 2.0	Node.js chaincode build environment

Source: https://github.com/hyperledger/fabric (Apache 2.0) **Production deployments:** IBM Food Trust, we.trade, TradeLens — demonstrating production-grade maturity.

---

2. Smart Contract (safety-chaincode)

Node.js packages used inside the `safetycc` chaincode that runs on the Fabric peer.

Package	Version	License	Role in solution
fabric-contract-api	^2.5.8	Apache 2.0	Provides the `Contract` base class, `ctx.stub`, and `ctx.clientIdentity` APIs used to read role attributes and write to the ledger
fabric-shim	^2.5.8	Apache 2.0	Low-level chaincode runtime — handles communication between the chaincode process and the peer over gRPC

Package source: https://www.npmjs.com/package/fabric-contract-api

---

3. IoT Gateway Simulation and Tamper Detection (safety-simulation)

Node.js packages used in the client application layer — the IoT gateway service, enrollment scripts, and tamper-detection watcher.

Package	Version	License	Role in solution
@hyperledger/fabric-gateway	^1.11.0	Apache 2.0	Fabric Gateway SDK — connects client applications to the peer, submits and evaluates transactions using enrolled X.509 identities
@grpc/grpc-js	^1.14.4	Apache 2.0	gRPC transport layer — handles the TLS-encrypted connection between the client and the peer's gateway endpoint
fabric-ca-client	^2.2.20	Apache 2.0	Fabric CA SDK — programmatically enrolls identities (gateway1, supervisor1) with the CA, receiving certificates and private keys into the local wallet
fabric-common	^2.2.20	Apache 2.0	Shared utilities used by fabric-ca-client
better-sqlite3	^12.11.1	MIT	Synchronous SQLite driver — stores raw TTN sensor payloads off-chain; also runs the AFTER UPDATE trigger that flags tampered records
uuid	^14.0.0	MIT	Generates unique incident IDs (e.g. INC-E621CE02) for each safety event

4. Infrastructure and Runtime

Component	Version	License	Role in solution
Docker Engine	27.x (via Docker Desktop)	Apache 2.0	Container runtime — all Fabric nodes run as Docker containers, simulating org-level network isolation
Docker Compose	v2.x	Apache 2.0	Defines and orchestrates all 7 containers (3 CAs, 2 peers, 1 orderer, 1 CLI) with correct networking, volumes, and environment
Node.js	v20 LTS	MIT	JavaScript runtime for chaincode, gateway service, simulation scripts, and tamper-detection watcher
Ubuntu 22.04 LTS (WSL2)	22.04	GPL/Various	Linux environment providing the bash shell, filesystem, and networking for all Fabric tooling
Git	2.34.x	GPL v2	Version control for all source code — network configuration, chaincode, and application scripts
SQLite	3.37.x	Public Domain	Embedded database for off-chain storage of raw sensor payloads

5. License compliance summary

License	Components	Commercial use permitted
Apache 2.0	All Hyperledger Fabric binaries and Docker images, fabric-contract-api, fabric-shim, @hyperledger/fabric-gateway, @grpc/grpc-js, fabric-ca-client, fabric-common, Docker	✅ Yes — explicit grant
MIT	better-sqlite3, uuid, Node.js	✅ Yes — permissive
GPL v2	Git	✅ Yes — tool use, not distributed

License	Components	Commercial use permitted
Public Domain	SQLite	✅ Yes — no restrictions

All components are open-source and permit commercial use. No proprietary or non-compliant dependencies are present. There are no GPL-licensed libraries linked into the application code (Git is a development tool, not a runtime dependency).

---

6. Original development vs open-source components

Per the challenge requirement to "clearly differentiate between proprietary development and third-party/open-source components":

Original development (startup's own work):

- `configtx.yaml` — IINoT network topology, MSP IDs, endorsement policies
- `docker/docker-compose.yaml` — IINoT-specific container configuration
- `scripts/network.sh` — custom network orchestration script
- `scripts/registerEnroll.sh` — identity registration with role attributes
- `safety-chaincode/lib/safetyContract.js` — all business logic: LogIncident, UpdateIncidentStatus, GetAllIncidents, GetIncidentHistory, role-based access control
- `safety-simulation/simulateEvent.js` — end-to-end IoT simulation
- `safety-simulation/watchAndVerify.js` — automated tamper detection watcher
- `safety-simulation/setupTriggers.sql` — SQLite trigger for instant tamper flagging
- `safety-simulation/enrollIdentities.js` — SDK-based identity enrollment
- `safety-simulation/getHistory.js` — audit trail query tool

Open-source components used as infrastructure/libraries (listed above in sections 1–4): All standard Hyperledger Fabric tooling, Node.js SDK packages, and system utilities. The `fabric-samples/test-network` was used only as a learning scaffold during initial development — the production network (`iinot-network`) is an independent custom implementation.

Document generated: June 2026 | Project: IINoT Lone Worker Safety Monitoring System